

JASC DATA PROCESSOR

Comprehensive User Manual

A lightweight, modular desktop application for data ingestion, visualization, mathematical transforms, statistics, and curve fitting.

Reference Manual & Guide

Version 0.1.0 (Stable release)
Compiled: June 2026
License: GNU General Public License v3.0

Table of Contents

1. Installation & Getting Started 3

System Requirements 3

Launching the Application 3

2. User Interface Layout 4

3. Data Ingestion 5

CSV/TXT File Import 5

Manual Spreadsheet Entry 5

Digitizing Data from Images (Import from Figure) 5

4. Data Management & Sheet Editing 7

Spreadsheet Cell Operations 7

Inserting & Deleting Rows and Columns 7

Column Metadata (Roles, Units, Long Names) 7

5. Mathematical Formulas & Expressions 8

Using the Apply Formula Dialog 8

Positional Aliases and Namespaces 8

6. Calculus, Signal Processing & Interpolation 9

Numerical Differentiation & Integration 9

FFT and Inverse FFT 9

Interpolation Workbench 9

7. Statistics Workbench 10

Single-Column Summaries & Diagnostics 10

Two-Column & Paired Tests 10

Grouped & Factor Analysis 10

Multi-Column PCA & Correlations 10

Companion Plots 10

8. Curve Fitting Engine 11

Standard Curve Fitting 11

Custom Mathematical Fit Models 11

Uncertainties & ODR (Orthogonal Distance Regression) 11

9. Visualization & Saved Plots 13

Plot Series Configuration 13

Axis Scaling, Ranges & Custom Styling 13

Saved Plot Definitions 13

10. Batch Processing Workbench **14**
Running Batch Operations 14
Execution Controls & Overwrite Safeguards 14
11. Troubleshooting & FAQs **15**

1. Installation & Getting Started

System Requirements

- Python 3.10 or higher.
- Dependencies listed in `requirements.txt` (including `PySide6`, `pandas`, `numpy`, `scipy`, `matplotlib`, `odrpack`, and `SciencePlots`).

Launching the Application

Navigate to the directory containing JASC in your terminal, and run:

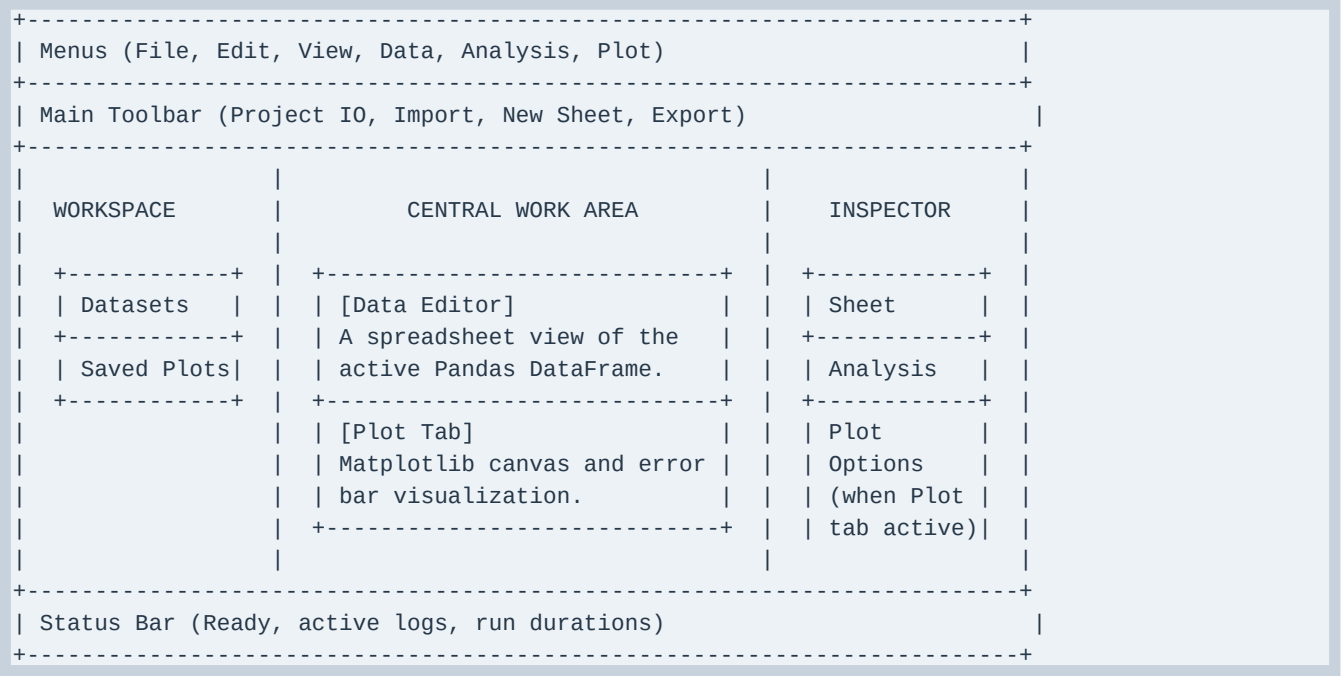
```
python main.py
```

This initializes the application window and opens an empty workspace.

---

2. User Interface Layout

JASC uses a **Docked Workspace Layout** where all secondary control panels dock neatly around the main work area.



- **Central Work Area**: Contains two tabs:

1. **Data**: Displays the selected sheet in a spreadsheet view.
2. **Plot**: Displays the active visualization canvas.

- **Workspace Panel (Left)**: Displays list of active **Datasets** and **Saved Plots**. Double-clicking items swaps active view.
- **Inspector Panel (Right)**: Shows contextual tools. When in the **Data** tab, it displays **Sheet** tools (add/delete/metadata) and **Analysis** actions. When in the **Plot** tab, it swaps to **Plot Options** (Series styling, Axes settings, and the Locator coordinates).
- **Resetting Layout**: If panels get closed or misplaced, select **View > Reset View to Default** (`Ctrl+Shift+R`) to restore the original layout.

---

3. Data Ingestion

CSV/TXT File Import

1. Select **File > Import Data...** (`Ctrl+I`) or click the **Import** button in the main toolbar.
2. In the dialog, customize:

- **Delimiter**: Comma, Tab, Semicolon, or Custom whitespace.
- **Headers**: Detect headers automatically or specify start row.

3. The dataset appears in the Workspace panel on the left with a unique name.

Manual Spreadsheet Entry

1. Select **File > New Blank Sheet** (`Ctrl+N`), click the toolbar icon, or click **New Sheet** at the bottom of the Datasets list.
2. Set the sheet size via the Data Inspector (**Sheet > Table Size**).
3. To generate data patterns, use **Sheet > Fill Range**. Specify:

- **Target Column**
- **Start Value** and **Step Size**
- **Row Range** and **Row Count**
- Click **OK** to populate arithmetic series (e.g. timestamps or independent variables).

Digitizing Data from Images (Import from Figure)

If you have a plot in an image (JPG/PNG) but need the underlying numbers, use JASC's built-in **Figure Digitizer**.

1. Launch it via **File > Import From Figure...** or the main toolbar button.
2. Load an image using **Load Image** on the top toolbar.

Before digitization, calibrate the pixel-to-data mapping:

1. Go to the **Calibration** tab.
2. Click **Pick X1** and click on a known value on the image's X-axis. Enter its calibrated value.
3. Repeat for **Pick X2**, **Pick Y1**, and **Pick Y2**.
4. Select whether the axis is **Linear** or **Logarithmic**.
5. Use the **Precision Zoom** panel with the magnifier enabled to click exact pixel centers.

- Select the **Manual** mode on the mode banner.
- Click anywhere on the plot lines to manually place points.
- To add error bars / uncertainties, click **Set Uncertainty** for a selected point, and click either a radial boundary or four coordinates representing error limits. Manual uncertainties are rendered as blue error bars.

If the plot has solid lines, let JASC trace it:

1. Set a painted region using the **ROI Brush** (translucent green mark) to restrict tracing.
2. Select **Curves/Lines > Auto Trace**.
3. Choose **Dark Pixels** or **Color Sample** (click a color in the image using the sampler).
4. Click **Run Auto Trace** to extract pixel paths.

For scatter plots:

1. Go to **Symbols**.
2. Click **Sample Symbol** and drag a bounding box around a symbol in the image.
3. JASC learns the template symbol, size, and color.
4. Select a **Symbol Shape Prior** (e.g., Circle, X cross, Plus cross, Square) to filter lines touching the symbols.
5. Click **Run Symbols Match**.
6. The digitizer extracts symbol positions along with pixel and calibrated uncertainty values based on symbol size.

Click **Create Dataset** to save your digitized coordinate table into the JASC workspace.

---

4. Data Management & Sheet Editing

Spreadsheet Cell Operations

- **Cell Editing**: Double-click any cell in the Data Editor to change its value. Press **Enter** to commit.
- **Undo/Redo**: Supported for all sheet changes (cell edits, deletes, formula applications, etc.). Press **Ctrl+Z** to Undo, and **Ctrl+Y** to Redo.
- **Clipboard Copy/Paste**:
- Select cells and press **Ctrl+C** to copy.
- Click on a destination cell and press **Ctrl+V** to paste.
- JASC automatically expands rows or columns to accommodate pasted blocks.

Inserting & Deleting Rows and Columns

Right-click on any cell or header in the Data Editor to access the spreadsheet context menu:

- **Insert Row / Column**: Adds a row/column before the selection.
- **Delete Row / Column**: Deletes selected rows or columns.
- **Whole-Column Clipboard**: Copy and paste entire columns, which preserves metadata (like column roles and units).

Column Metadata (Roles, Units, Long Names)

Double-click a column header (e.g. `col_0`) to open the **Column Metadata** dialog.

- **Name**: The programmatic column identifier used in formulas (must be alphanumeric/no spaces).
- **Role**: Select `x`, `y`, `z`, `x_error`, `y_error`, `group`, `label`, or `weight`.
- **Long Name** & **Unit**: Used to automatically prefill Matplotlib plot titles and axis labels.
- **Comments**: Document column origin or transformations.

---

5. Mathematical Formulas & Expressions

Apply custom expressions directly in columns.

Using the Apply Formula Dialog

1. Select **Analysis > Apply Formula...** (`Ctrl+Shift+F`) or click the inspector button.
2. Select the target column (or enter a new name to create one).
3. In the formula text box, type your expression (e.g., `col("y") * 2.5` or `col1 / col2`).
4. Select the cell scope: **All Rows** or **Selected Rows Only**.
5. Click **Run** to execute.

Positional Aliases and Namespaces

- **Positional Aliases**: You can refer to columns by their index, like `col1` (first column), `col2` (second column), etc., as an alternative to writing the exact name `col("Column Name")` .
- **NumPy/SciPy Namespace**: JASC sandbox imports standard mathematical packages. You can write expressions using:
  - `np.sin()` , `np.log10()` , `np.exp()` , `np.sqrt()` , `np.pi`
  - `special` (SciPy special functions)
  - `stats` (SciPy stats distributions)
  - `np.random.normal()` (NumPy random values)

> [WARNING] Standard Python power syntax is `**` (e.g. `col1 ** 2`). Do not use `^` as JASC blocks it to prevent common XOR bitwise operation bugs.

---

6. Calculus, Signal Processing & Interpolation

Numerical Differentiation & Integration

Available via the **Analysis** menu or the Data Inspector:

- **Derivative**: Select X and Y columns. Choose from methods: *NumPy gradient*, *Forward difference*, *Backward difference*, *Central difference*, or *Savitzky-Golay* smoothing derivative.
- **Integral**: Select X and Y columns. Choose from integration algorithms: *Trapezoid*, *Simpson*, *Left*, *Right*, or *Midpoint* rectangle. You can specify an **Initial Value** offset (integration constant) to align the integrated curve with physical bounds.

Select **Plot result immediately** to automatically view the calculus output.

FFT and Inverse FFT

- **FFT (Fast Fourier Transform)**: Computes the spectral amplitude and frequency. Select a signal column, sampling interval, and a window function (*Rectangular*, *Hann*, *Hamming*, *Blackman*, *Bartlett*, *Flat top*). JASC creates a new sheet containing the frequency spectrum.
- **iFFT (Inverse FFT)**: Reconstructs time-domain signals. Select the real and optional imaginary spectrum columns.

Interpolation Workbench

Launch via **Analysis > Interpolation...** to predict values at arbitrary points:

- **1D Interpolation**: Linear, nearest neighbor, cubic spline, PCHIP, Akima, or smoothing splines.
- **2D Interpolation**: Grid-based or scattered data methods (linear, cubic, RBF).
- **N-D Interpolation**: Radial Basis Functions (RBF) or nearest-neighbor queries.
- **Query Coordinates**: Generate a query sequence automatically, select an existing column, or enter coordinates manually.

---

7. Statistics Workbench

Perform statistical evaluations directly inside the JASC environment. Access these under **Analysis > Statistics...**

Single-Column Summaries & Diagnostics

Returns basic descriptive parameters (Mean, Std Dev, Variance, SEM, Skewness, Kurtosis), percentiles, normality tests (Shapiro-Wilk, Anderson-Darling), and outlier checks (Z-score thresholds, IQR).

Two-Column & Paired Tests

Evaluate relationships between columns:

- **Correlations**: Pearson, Spearman, and Kendall coefficients.
- **Comparison Tests**: Paired t-tests, independent-samples t-tests, and Mann-Whitney U tests.
- **Agreement**: Bland-Altman statistics and linear regressions.

Grouped & Factor Analysis

If you have a factor or group column (e.g. categorical variables) and a value column:

- Summarizes statistics per group.
- Runs **One-way ANOVA** or **Kruskal-Wallis** tests.
- Performs **Tukey HSD post-hoc comparisons** to isolate pairwise differences.

Multi-Column PCA & Correlations

Analyze multiple variables in a single operation:

- Computes pairwise correlation matrices.
- Performs **Principal Component Analysis (PCA)**, reporting eigenvalue distribution, explained variance, loading vectors, and PC scores.

Companion Plots

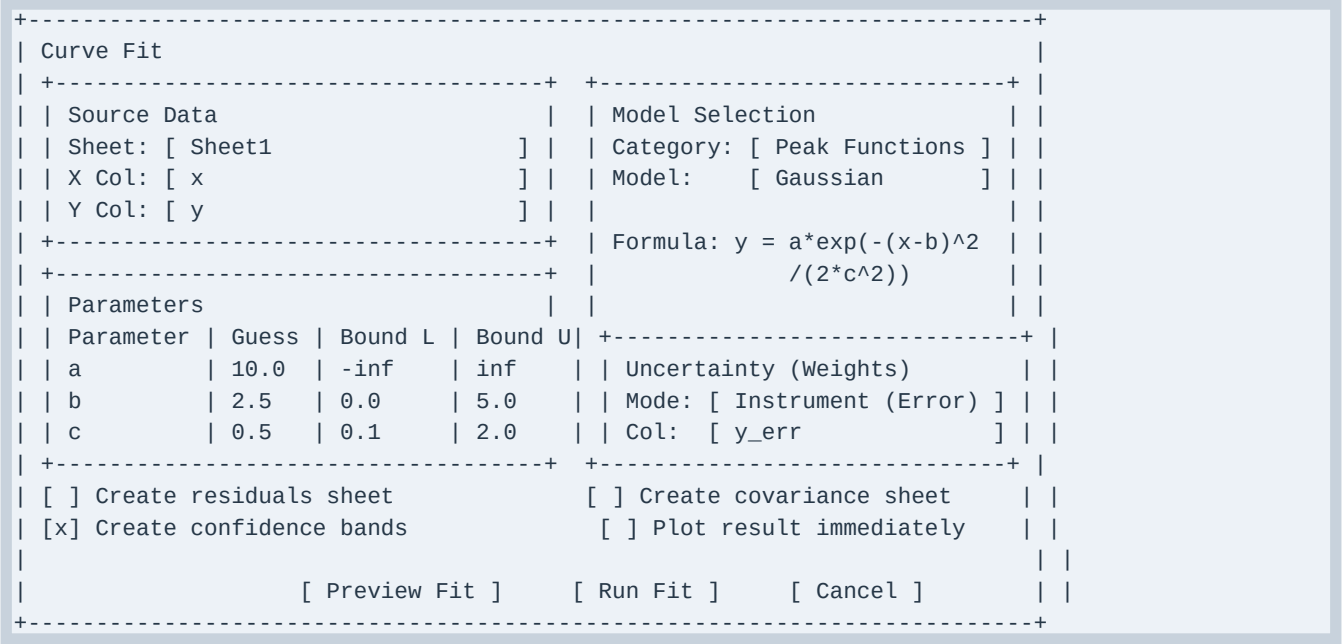
Check **Create companion plot** to generate Matplotlib visualizations linked to your stats:

- Normality histograms with normal distribution fits.
- Grouped box/violin comparison charts.
- Bland-Altman bias/limit charts.
- PCA scree plots.

---

8. Curve Fitting Engine

JASC provides a curve fitting engine for analytical models. Open via **Analysis > Curve Fit...** (`Ctrl+Shift+C`) .



Standard Curve Fitting

1. Select the source Sheet, X, and Y columns.
2. Select a category (e.g. Linear, Exponential, Trigonometric, Peak Functions) and a specific model (e.g. Gaussian, Heidler, Bessel, Lorentz).
3. Set initial parameter guesses, lower and upper bounds, or check **Fixed** to hold a parameter constant.
4. Select fit algorithms: *Levenberg-Marquardt*, *Trust Region Reflective*, or *Dogbox*.
5. Click **Run Fit**. JASC displays a **Fit Report** (with fit parameters, standard errors, covariance, R^2, and residuals) and plots the fit curve over your raw data.

Custom Mathematical Fit Models

Create custom fitting equations via **Analysis > Custom Fit Models...**:

1. Click **Add Model**.
2. Define parameters as a comma-separated list (e.g. `a, b, c`).
3. Write the math expression using safe Python/NumPy (e.g. `a * np.sin(b * x + c)`).
4. Save the model. It becomes available in the standard **Curve Fit** model selector under the *Custom* category.

Uncertainties & ODR (Orthogonal Distance Regression)

- **Y Uncertainty**: Select a column representing Y measurement error to run weighted fitting.
- **X/Y Uncertainty (ODR)**: If your independent variable X also has errors, select an X-error column. JASC automatically swaps the solver to **Orthogonal Distance Regression (ODR)** to calculate correct parameters accounting for errors in both dimensions.

---

9. Visualization & Saved Plots

Plot Series Configuration

1. Select the **Plot** tab, then view the **Plot Inspector** on the right.
2. Click **Add Series** in the Series tab.
3. Assign columns to X, Y (and Z/weights if applicable).
4. Select a **Plot Type**:

- *2D Line & Scatter*: Line, Scatter, Line + Markers, Errorbar.
- *Distribution & Reporting*: Box Plot, Violin Plot, Histogram, Bar (Vertical/Horizontal), Area/Fill.
- *Composition*: Pie, Donut.
- *2D Surface Maps*: Interpolated Heatmap, Contour, Filled Contour.

5. Set styles: line color, line width, markers, fills, and alphas.

Axis Scaling, Ranges & Custom Styling

- Go to the **Axes** tab in the Plot Inspector:
- Manually set limits for X, left Y, and right Y axes, or leave them blank to auto-scale.
- Swap scales between **Linear** and **Logarithmic**.
- Toggle the plot grid on/off.
- Go to the **Style** tab:
- Select **SciencePlots journal templates** (e.g. `science`, `nature`, `ieee`).
- Enable/disable LaTeX formatting.
- Adjust output figure dimensions.

Saved Plot Definitions

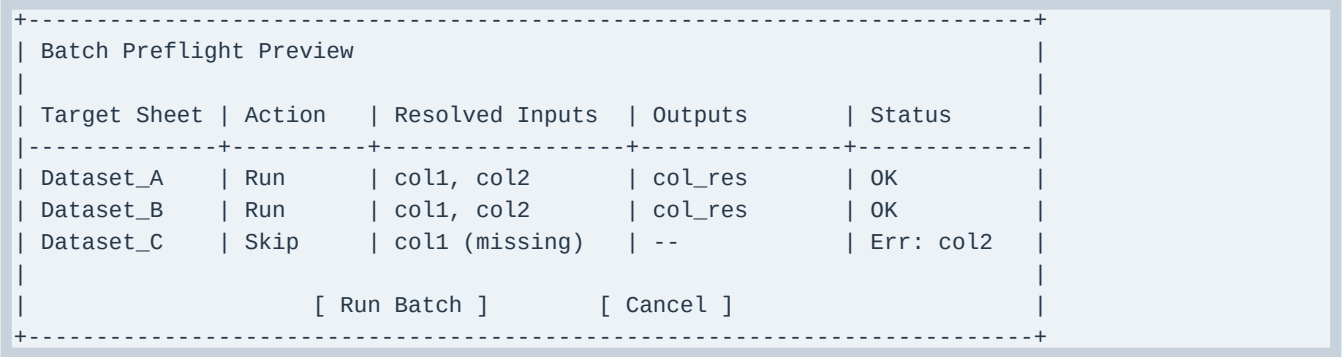
Once your plot looks correct, save its state so you can return to it later:

1. Click **Save Plot** at the top of the Plot tab or select **Plot > Save Current Plot...**
2. Name the plot definition.
3. The plot appears in the **Plots** tab in the Workspace panel.
4. Double-clicking any item in this list restores the exact series, styling, axes, scales, and grids.

---

10. Batch Processing Workbench

JASC's most powerful productivity feature is the **Batch Processing Workbench**, which lets you repeat calculations across multiple worksheets.



Running Batch Operations

Every main operation dialog (Formula, Derivative, Integral, FFT, Interpolation, Statistics, Curve Fit) supports batch execution:

1. In the operation dialog, expand the **Target Worksheets** list at the bottom.
2. Select the sheets you want to target.
3. Select the **Column Binding Mode**:

- **Exact Name**: Matches columns by exact text headers.
- **Column Position**: Matches columns by their layout order (e.g. `col1` , `col2`).

4. Click **Run**. JASC displays a **Batch Preflight Preview** listing target sheets, mapping resolutions, outputs, and any compatibility errors.
5. Confirm and run. A cancelable modal progress bar shows the execution state.

Execution Controls & Overwrite Safeguards

- **Overwrite Safeguards**: In batch formulas or calculus, JASC defaults to creating new unique output columns to prevent accidental data loss. You can explicitly confirm an overwrite update policy in the preflight check.
- **Batch Summaries**: Multi-sheet calculations generate a cumulative summary sheet (e.g., `{active_sheet}_fit_batch_summary` or `{active_sheet}_statistics_batch_summary`). These summary sheets aggregate parameter fits, R^2, averages, test metrics, and execution logs in a tidy table format ready for plotting or exporting.
- **Parallel Workers**: JASC utilizes process-based multiprocessing for headless batch sheets, running operations in parallel across CPU cores without locking the active GUI.

---

11. Troubleshooting & FAQs

Make sure you are executing python from the active conda environment:

```
& "C:\Users\julia\anaconda3\envs\dataProcessing\python.exe" main.py
```

JASC detects when a worksheet has more than 100,000 cells or 5MB memory footprint. The Data Editor limits automated column autosizing to prevent GUI lag. For very large datasets, consider using external downsampling or editing in chunks.

- Check if your initial guesses are realistic.
- Verify if bounds are too narrow or overlap.
- Swap from *Levenberg-Marquardt* to *Trust Region Reflective* to respect parameter bounds.

To save a sheet as a CSV or TSV file, select the sheet, open the Data Inspector, and click **Export** (or select **File > Export Dataset...**). To export figures, click the save floppy disk icon on the Matplotlib toolbar in the Plot tab.